

```
#!/usr/bin/env python3
"""
```

```
npu_manager.py
```

```
-----
```

PURPOSE FOR BEGINNER DEVELOPERS:

This daemon is the "traffic cop" for mobile AI inference on the Snapdragon NPU.

Because phones have strict RAM limits (e.g. 16GB total, 8-11GB usable for AI), you CANNOT load multiple LLMs into memory at the same time. If you do, Android's Low Memory Killer (LMK) will crash the whole system.

This script:

1. Listens on a standardized UNIX Domain Socket (/dev/socket/npu_manager.sock).
2. Gracefully unloads the active model from NPU/RAM before loading the next one.
3. Knows how to execute the three different backends:
 - GenieX (for QAI GGUFs)
 - llama-server (for Unsloth GGUFs)
 - qnn-net-run (for Gemma 4 QAT .bin contexts)
4. Exposes an HTTP port for OmniRoute to route prompt tokens into.

```
"""
```

```
import os
import sys
import time
import socket
import subprocess
import signal
import json
from pathlib import Path
```

```
SOCKET_PATH = "/dev/socket/npu_manager.sock"
```

```
MODELS_DIR = Path("/data/local/tmp/models")
```

```
CURRENT_PROCESS = None
```

```
CURRENT_MODEL = None
```

```
MODEL_REGISTRY = {
```

```
    "qai-qwen-9b": {
        "type": "geniex",
        "path": MODELS_DIR / "qai" / "qwen3.5_9b_q4_0.gguf",
        "cmd": ["geniex", "server", "--model", "{path}", "--device", "hybrid", "--port", "8080"]
    },
    "unsloth-custom": {
        "type": "llama_cpp",
        "path": MODELS_DIR / "unsloth" / "custom_task_q4_k_m.gguf",
```

```

        "cmd": ["llama-server", "-m", "{path}", "--threads", "4", "--port", "8080"]
    },
    "gemma4-qat": {
        "type": "qnn_native",
        "path": MODELS_DIR / "qnn_qat" / "gemma4_int4_context.bin",
        "cmd": ["qnn-server", "--backend", "libQnnHtp.so", "--context", "{path}", "--port", "8080"]
    }
}

```

```

def unload_current_model():
    global CURRENT_PROCESS, CURRENT_MODEL
    if CURRENT_PROCESS:
        print(f"[*] Unloading active model: {CURRENT_MODEL} (PID: {CURRENT_PROCESS.pid})")
        CURRENT_PROCESS.send_signal(signal.SIGTERM)
        try:
            CURRENT_PROCESS.wait(timeout=5)
        except subprocess.TimeoutExpired:
            print("[!] Process did not exit cleanly. Forcing SIGKILL...")
            CURRENT_PROCESS.kill()
        CURRENT_PROCESS = None
        CURRENT_MODEL = None
        time.sleep(0.5)
        print("[✓] Memory cleared.")

```

```

def load_model(model_key: str) -> bool:
    global CURRENT_PROCESS, CURRENT_MODEL
    if model_key not in MODEL_REGISTRY:
        print(f"[!] ERROR: Unknown model key '{model_key}'")
        return False

```

```

    config = MODEL_REGISTRY[model_key]
    model_path = config["path"]
    cmd = [arg.replace("{path}", str(model_path)) for arg in config["cmd"]]

```

```

    print(f"[*] Spawning {config['type']} backend: {' '.join(cmd)}")
    try:
        CURRENT_PROCESS = subprocess.Popen(
            cmd,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            preexec_fn=os.setsid
        )

```

```

CURRENT_MODEL = model_key
time.sleep(2)
print(f"[✓] Successfully loaded {model_key} on http://127.0.0.1:8080")
return True
except Exception as e:
    print(f"[!] Failed to launch model process: {e}")
    return False

def run_socket_server():
    if os.path.exists(SOCKET_PATH):
        os.remove(SOCKET_PATH)

    server = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
    server.bind(SOCKET_PATH)
    os.chmod(SOCKET_PATH, 0o666)
    server.listen(5)
    print(f"[*] NPU Manager listening on UNIX socket: {SOCKET_PATH}")

    while True:
        conn, _ = server.accept()
        raw_data = conn.recv(1024).decode("utf-8")
        if not raw_data:
            conn.close()
            continue

        try:
            req = json.loads(raw_data)
            action = req.get("action")
            target = req.get("target")

            if action == "SWAP":
                unload_current_model()
                success = load_model(target)
                response = {"status": "OK" if success else "ERROR", "current_model":
CURRENT_MODEL}
            elif action == "STATUS":
                response = {"status": "RUNNING", "current_model": CURRENT_MODEL}
            elif action == "UNLOAD":
                unload_current_model()
                response = {"status": "UNLOADED", "current_model": None}
            else:
                response = {"status": "INVALID_ACTION"}

```

```
        conn.sendall(json.dumps(response).encode("utf-8"))
    except Exception as e:
        err_resp = {"status": "EXCEPTION", "error": str(e)}
        conn.sendall(json.dumps(err_resp).encode("utf-8"))
    finally:
        conn.close()

if __name__ == "__main__":
    try:
        run_socket_server()
    except KeyboardInterrupt:
        print("\n[*] Shutting down NPU Manager...")
        unload_current_model()
        if os.path.exists(SOCKET_PATH):
            os.remove(SOCKET_PATH)
```